

GitLab CI/CD with Automated Testing and JAR Packaging

Aim

To implement an automated CI/CD pipeline using **Git, GitLab, GitLab CI/CD, Maven, and automated testing**, where the application is automatically built, tested, packaged into a JAR file, and stored as a GitLab artifact.

Objective

Students will learn to:

1. Create a Java Maven project.
2. Create a Git repository.
3. Push the project to GitLab.
4. Configure GitLab CI/CD.
5. Automatically compile the application.
6. Automatically execute test cases.
7. Stop the pipeline when tests fail.
8. Package the application into a JAR file.
9. Store the JAR as a GitLab artifact.

Procedure

Step 1 — Create App.java

Create: `src/main/java/App.java`

```
public class App {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("GitLab CI/CD Application");  
    }  
}
```

Step 2 — Create AppTest.java

Create: `src/test/java/AppTest.java`

Use JUnit:

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.assertEquals;  
  
public class AppTest {
```

```

@Test
public void testAddition() {
    assertEquals(5, App.add(2, 3));
}
}

```

Step 3 — Create pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>gitlab-cicd-demo</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
            </plugin>
        </plugins>
    </build>
</project>

```

Step 4 — Create .gitignore

Create: `.gitignore`

Put:

```
target/
```

Step 5 — Initialize Git

Open Git Bash inside the project folder.

```
cd Desktop/GitLab-CICD-Demo
```

```
# Initialize Git
git init

# Check status
git status

# Add and commit
git add .
git commit -m "Create Java application with automated test"
```

Step 6 — Connect Git to GitLab

a. Create a repo on GitLab

- Log into GitLab → click **New project** → **Create blank project**
- Give it a name (e.g. `gitlab-cicd-demo`) → set visibility → do **not** initialize with a README (avoids merge conflicts with your local repo) → **Create project**
- GitLab shows you a repo URL like `https://gitlab.com/YOUR_USERNAME/gitlab-cicd-demo.git`

b. Link the local repo to GitLab

```
# Add the GitLab repository
git remote add origin https://gitlab.com/YOUR_USERNAME/gitlab-cicd-demo.git

# Check
git remote -v

# Change the branch to main
git branch -M main

# Push
git push -u origin main
```

c. Authentication note

GitLab has phased out plain password auth over HTTPS. When prompted for a password on push, use a **Personal Access Token** instead:

- GitLab → **Edit profile** → **Access Tokens** → create a token with `write_repository` scope
- Use your username + that token as the password when prompted

Alternatively, set up SSH keys and use an SSH remote URL instead of HTTPS:

```
git@gitlab.com:YOUR_USERNAME/gitlab-cicd-demo.git
```

Step 7 — Create GitLab CI/CD Pipeline

Create: `.gitlab-ci.yml`

```
image: maven:3.9-eclipse-temurin-21

stages:
  - build
  - test
  - package
```

```
build:
  stage: build
  script:
    - mvn compile

test:
  stage: test
  script:
    - mvn test

package:
  stage: package
  script:
    - mvn package
  artifacts:
    paths:
      - target/*.jar
    expire_in: 1 week
```

Step 8 — Push the Pipeline

```
git add .gitlab-ci.yml
git commit -m "Add GitLab CI automated testing pipeline"
git push
```

Step 9 — Observe the Pipeline

Go to your GitLab project.

Select: **Build → Pipelines**

You should see:

```
Pipeline #1
build    ✓
test     ✓
package  ✓
```

Step 10 — Download the JAR

Open the successful `package` job.

Find: **Job artifacts**

Download it. You will find:

```
target/
gitlab-cicd-demo-1.0.jar
```

Result

The GitLab CI/CD pipeline was successfully implemented for the Java Maven application. The source code was pushed to the GitLab repository, and the `.gitlab-ci.yml` configuration

automatically triggered the CI/CD pipeline.